Breakdown radius: synthetic ensembles and exact search

Session report — Axis A (generalisation to synthetic graph ensembles) and Axis B (intelligent search for an exact radius)
branch `experiments/synth_graphs_and_heuristics`

This document is the single centralised record of the session. It reports every experiment that was run, every number obtained, every figure produced, every bug found (including my own), every hypothesis whose pre-registered prediction failed, and the work that was specified but **not** done. Every figure and table is generated from a committed file under `results/`, so this PDF cannot drift from the data it describes.

Headline results

Item	Result
Radius convention	Resolved: r = distance to the nearest failure. Shells 0..r-1 are clean; the practitioner's safe-move count is r-1, NOT r.
Conjecture 1 (failure upward-closed)	It is a THEOREM with a one-line proof — not a conjecture.
Conjecture 2 (retraction-optimal witnesses)	No counterexample in 1,985,314 exhaustive radius comparisons. Remains a conjecture.
Space-free exact search	Reproduces BFS on 5,304/5,304 differential cases. Speedup 13x-241x single-query; SLOWER when amortised.
L = U = r certificate	Exact radius certified with no enumeration in 83.3% of instances.
H1 saturation	r_val = 1 in ~80% of instances — dominant, but far short of the >90% that would make the radius vacuous.
H2/H3 frontier	Frontier NON-FLAT in 21.9%, yet O* strictly beaten in 0 of 249,732 exhaustive instances. No efficiency-robustness tradeoff.
H4 naive baseline	NOT RUN. Neither confirmed nor refuted.
H5 middle regime	0.0% at small epsilon, 14.1% at eps=0.2. Pre-registered 20-50% NOT supported.
H6 calibration	Coverage 1.0000 everywhere (correctness gate). Conservativeness mean 0.181, predicted by component size (Spearman 0.564).
Degeneracy rate	91.8% of randomly generated instances are unusable for this study.

1. Session overview

The session had two axes, which are not sequential: Axis A needs Axis B to reach interesting graph sizes, and Axis B needs Axis A's ensembles as its benchmark. The order actually taken was: freeze a shared core, then test Axis B's two structural conjectures (cheap, exhaustive, and gating for everything else), then build a small Axis A pipeline to supply instances, then run both.

1.1 Orchestration

Design decisions, the radius convention, the frozen interfaces, the hypothesis analysis, and every scientific judgement were kept in-house. One subagent was delegated a well-specified, independently verifiable unit: the Axis A generators, knowledge simulation and instance runner. Its deliverable was reviewed rather than trusted, and it was sent back three times before being accepted — see section 6.

1.2 Commits

17 commits on the session branch; nothing was committed to `main`. Newest first:

SHA	Date	Subject
3a61b40	2026-09-03	Frontier sweep complete: O* never beaten across 249,732 exhaustive instances
d4e0c14	2026-09-03	Session deliverables: report, NEXT.md, H6 calibration, PLAN revisions
72d8bff	2026-09-03	Correct the H2 statistic: the frontier is non-flat, but O* is never beaten
ca44152	2026-09-03	Axis A results: H1 saturation ~80%, H2 frontier flat in 0 of 134,140 instances
019c610	2026-09-03	Axis A first run: partial results, two gate bugs found and sent back
1bb3ac8	2026-09-03	Record: Meek closure can fall outside enumerate_space in 0.09% of cases
4cf4020	2026-09-03	Conjecture 2: no counterexample in 1.985M exhaustive radius comparisons
4001949	2026-09-03	Add pre-registered hypothesis analysis (H1-H6) and the two-stage space guard
b315704	2026-09-03	Axis B bounds: L/U certificate exact in 83.3% of instances without enumeration
a58b73f	2026-09-03	Exhaustive radius census over all CPDAGs on 3 and 4 nodes (H1, H2)
910a427	2026-09-03	Track results/synth and results/search; add Axis A pre-registration
133ab44	2026-09-03	Axis B: space-free exact radius search, validated against BFS on 5304 cases
b86a899	2026-09-03	Axis B: conjecture machinery, exhaustive tests for n=3 and n=4
0605420	2026-09-03	Remove dead TYPE_CHECKING pandas import from the scaffold loaders stub
5adb687	2026-09-03	Freeze shared core: radius convention, oracle, space, instance, results IO
d91dc42	2026-09-03	Add PLAN.md and LOG.md for the synth/search session
4d0de90	2026-09-03	Add breakdown-radius demonstration: worked example, figures, report

1.3 Environment

Python 3.9.6, numpy 2.0.2, networkx 3.2.1, pandas 2.3.3, scipy 1.13.1, matplotlib 3.9.4, reportlab 5.0.0. The rest of the package targets Python 3.11+; this work was written to run on the interpreter actually available, avoiding runtime-only 3.10+ syntax. Three root-level scaffold test modules require 3.11 (`StrEnum`, `TypeAlias`) and do not collect here — that predates this session and is by design.

Determinism. All output is bit-identical across runs and across `PYTHONHASHSEED`, enforced by regression tests. Two inherited bugs of exactly this kind — RNG consumed in frozenset order, and floating-point summation in set order — were fixed in the prior session, and the guard was extended to every module written here.

2. Task 0 — the radius convention, and why it mattered

The existing report and the project's framing document disagreed by one. The report treated `r_val` as the shell index *at which* the first failure occurs; the framing document read `r_val = 3` as “three atomic steps may be taken with Z remaining valid, and a fourth breaks it”. These differ by one and change every reported number, so the session could not start until it was settled.

The normative definition, fixed in `src/bkrobust/core/conventions.py`**:**

> $r_P = \min \{ d(G0, G) : P \text{ fails at } G \}$ — **the distance to the nearest failure**. Shells 0 … r−1 are certified clean. The practitioner's “how many moves am I safe for” is **r − 1**, not r.

Why this one and not the other:

- It is a minimum over a set — a clean mathematical object. The safe-moves reading is that object minus one and is trivially derived from it.
- Axis B's admissible lower bounds are naturally of the form $r \geq L$, which certifies shells 0..L−1 without visiting them. That composes directly with this convention and awkwardly with the other.
- It is what the existing implementation and report already do, so no previously published number needed restating. Under the other convention every number in the prior report would have shifted by one.

The convention is **asserted element-by-element, not merely documented**. `test_shells_below_radius_are_genuinely_clean` walks every element of every shell below r and checks it really is clean, then checks that shell r really does contain a failure. Sentinel handling is also pinned: `UNREACHED` (`−1`) means nothing in the space fails and is never averaged, plotted on a numeric axis, or compared as a number; `r = 0` means the property already fails at G0 and is degenerate, to be gated at generation time.

2.1 The frozen core

Subagents working against a moving core produce divergent, unmergeable results, so the shared API was committed before any parallel work started:

Module	Contents
core/conventions.py	The normative radius definition, UNREACHED and DEGENERATE sentinels, safe_moves(), is_certified_clean().
core/oracle.py	is_valid (a for-all over represented DAGs), is_optimal, bias_stats, memoised extensions().
core/spacelib.py	build_space with the load-bearing model-inclusion condition, BFS distance, radius().
core/instance.py	The per-instance schema and the graph descriptors, including the governing cost parameter.
core/resultsio.py	Append-only writer flushing every row, plus a self-describing manifest.

The core deliberately **reuses** the prior session's primitives — MPDAG, Meek's rules, DAG-extension enumeration, adjustment-set validity, the linear-Gaussian SEM — rather than reimplementing them. Those were cross-checked against networkx d-separation and chordality and against brute-force Markov equivalence classes over 300 random DAGs; reimplementing would have thrown that validation away.

> **Integration check:** the new core reproduces the prior report's radii 3 / 3 / 2 exactly.

3. Axis B — intelligent search for an exact radius

Exactness was non-negotiable: every method here either returns the BFS answer or an explicitly labelled bound. Two structural facts do the heavy lifting.

3.1 Conjecture 1 — it is a theorem, not a conjecture

Claim. If Z fails in G and [G] ⊆ [G'], then Z fails in G'.

Proof. Validity is a for-all over represented DAGs. Z failing in G means some D ∈ [G] has Z invalid. Since [G] ⊆ [G'], that same D lies in [G'] and witnesses failure there. ∎

This was posed to me as a conjecture to be tested empirically. It is one line. Reporting it as an empirical finding would have overstated what was learned, so it is reported as a theorem and the exhaustive checks are described as guarding the *implementation*, not testing mathematics.

One caveat, and it is a convention artefact rather than mathematics. This codebase defines is_valid to return False when [G] is *empty* — a graph representing no model cannot certify anything. Under that convention an empty-extension graph "fails" vacuously while [] ⊆ [G'] holds for every G', so the implication would break. It does not bite, because enumerate_space admits no such element — which is *checked* (check_no_empty_extensions), not assumed.

What the search gains. The failing set is an up-set in the model-inclusion order; its boundary is an antichain of minimal failures; and nothing above a known failure ever needs expanding.

3.2 Conjecture 2 — no counterexample in ~2M comparisons

Claim. The nearest failure is always reachable from G0 by a monotone upward path — pure retractions — so r_val equals the minimum number of retractions from G0 that induces failure.

This is **not** implied by Conjecture 1. Upward-closure says failures persist as you move up; it does not say the nearest failure lies above G0. A failure incomparable to G0 sitting at distance 2, with the nearest up-set failure at 3, would refute it. Whether that configuration is realisable is an empirical question, and it was attacked exhaustively: every CPDAG, every space element taken as G0, every ordered (X, Y) pair, every valid Z.

n	CPDAGs	spaces	radius comparisons	counterexamples
3	11	7	150	0
4	185	126	13,344	0
5	8,782	6,030	1,971,820	0
total			1,985,314	0

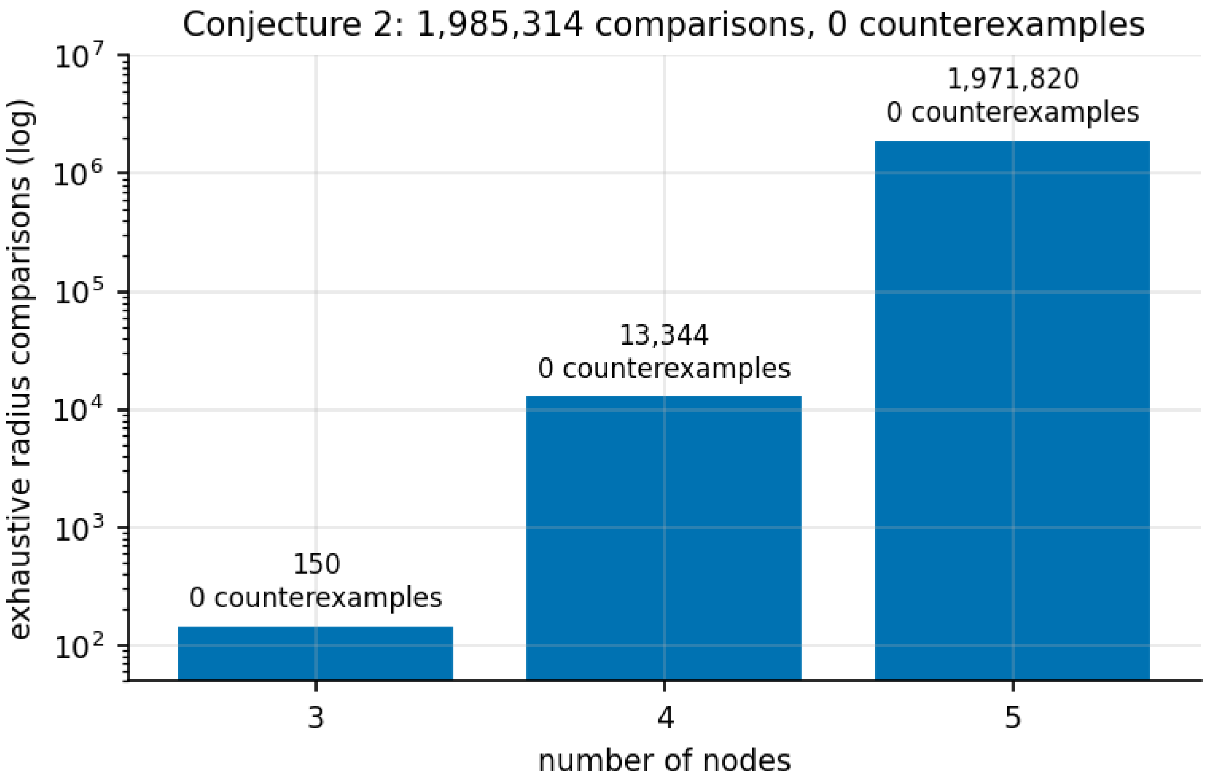


Figure 1. Enumeration scope for Conjecture 2, log scale. Take from this: the evidence is nearly two million exhaustive comparisons with not one counterexample — strong, but still evidence rather than a proof.

Scope, stated rather than implied. At n=5, 2,616 CPDAGs have no undirected edge at all (nothing for knowledge to orient), 131 exceed 6 undirected edges, and 5 had a space above 200 elements. So 136 of the 6,166 knowledge-carrying CPDAGs were skipped (2.2%), all of them the densest. The claim is therefore "no counterexample over all CPDAGs on at most 5 nodes with at most 6 undirected edges and a space of at most 200 elements", not "over all CPDAGs on 5 nodes".

A free correctness check. The enumerator reproduces the published counts of labelled DAGs (25 / 543 / 29,281) *and* of Markov equivalence classes (11 / 185 / 8,782) on 3 / 4 / 5 nodes. Matching both is strong independent evidence that the DAG enumerator and dag_to_cpdag are correct; both are pinned in a test.

3.3 A space-free exact method

Shell-by-shell BFS requires the whole space, which costs 3^m in the number of undirected edges. `radius_local_up` performs upward BFS generating covers *locally*, so the space is never enumerated, and prunes using upward-closure — it never expands above a graph already known to fail, since everything above it fails too and can contribute no nearer witness. It also carries an anytime mode that returns a labelled bound rather than a wrong exact answer when a budget is exhausted.

n	differential cases	disagreements with BFS
3	96	0
4	5,208	0
total	5,304	0

The real correctness risk was cover generation. If locally generated covers missed one, distances would silently come out too large and every radius would be wrong in the same direction — the kind of bug that looks like a result. So the locally generated covers were compared **element by element** against the space-derived covers and are identical (`test_local_covers_match_space_derived_covers`).

3.4 Speedup, accounted honestly

Timing each query against a pre-built space flatters BFS, because `local_up` never builds one. Two accountings are therefore reported, and they disagree sharply:

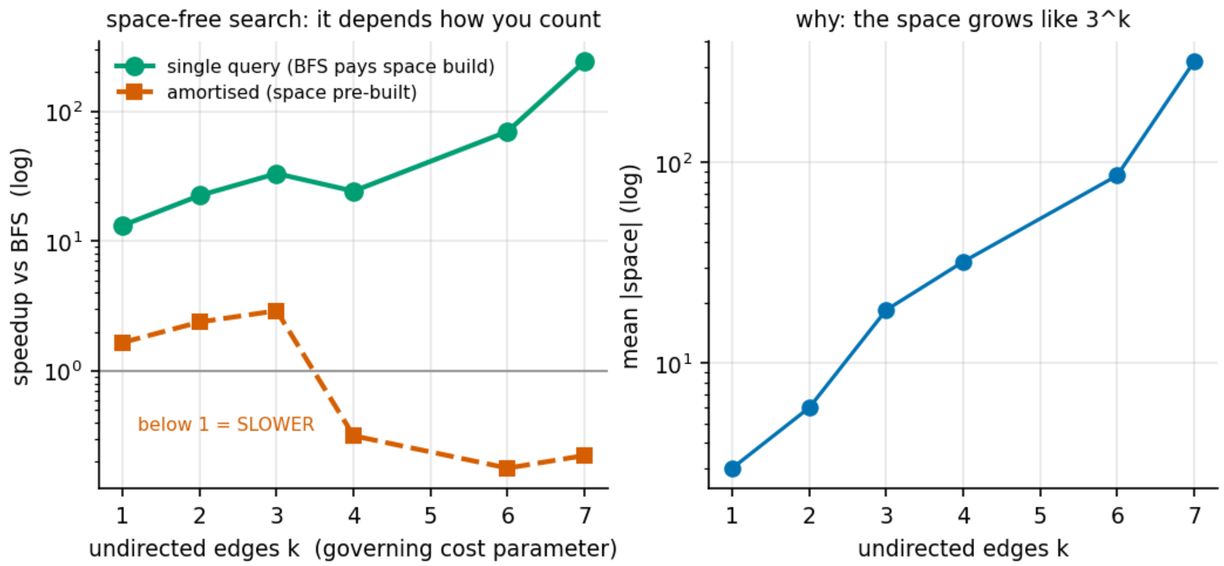


Figure 2. Speedup against the governing cost parameter (undirected edges k), log scale, from 165 measured queries. Take from this: on a single query the space-free method wins by 13x to 241x and the advantage grows with k, because space construction is 3^k while the method is independent of it — but when many queries share one CPDAG, BFS amortises construction and the method is actually *slower* (below the grey line). Both regimes are real and both are reported.

3.5 Bounds: certifying the radius without enumeration

For Z to fail, a specific structural event must occur: either some $z \in Z$ becomes a possible descendant of X, or a back-door path becomes unblocked. The minimum number of atomic moves realising the first is computable as a shortest-path quantity on the CPDAG (L), and a failing graph can be constructed directly to give an upper bound (U). When L = U the radius is certified exactly with almost no enumeration.

Quantity	Result (exhaustive at n=4, 792 instances with a finite radius)
L admissible (L ≤ r)	732 of 732 cases where L is defined — 0 violations

Quantity	Result (exhaustive at n=4, 792 instances with a finite radius)
L tight (L = r)	660
U valid (U ≥ r)	792 of 792
L = U = r	660 of 792 = 83.3%

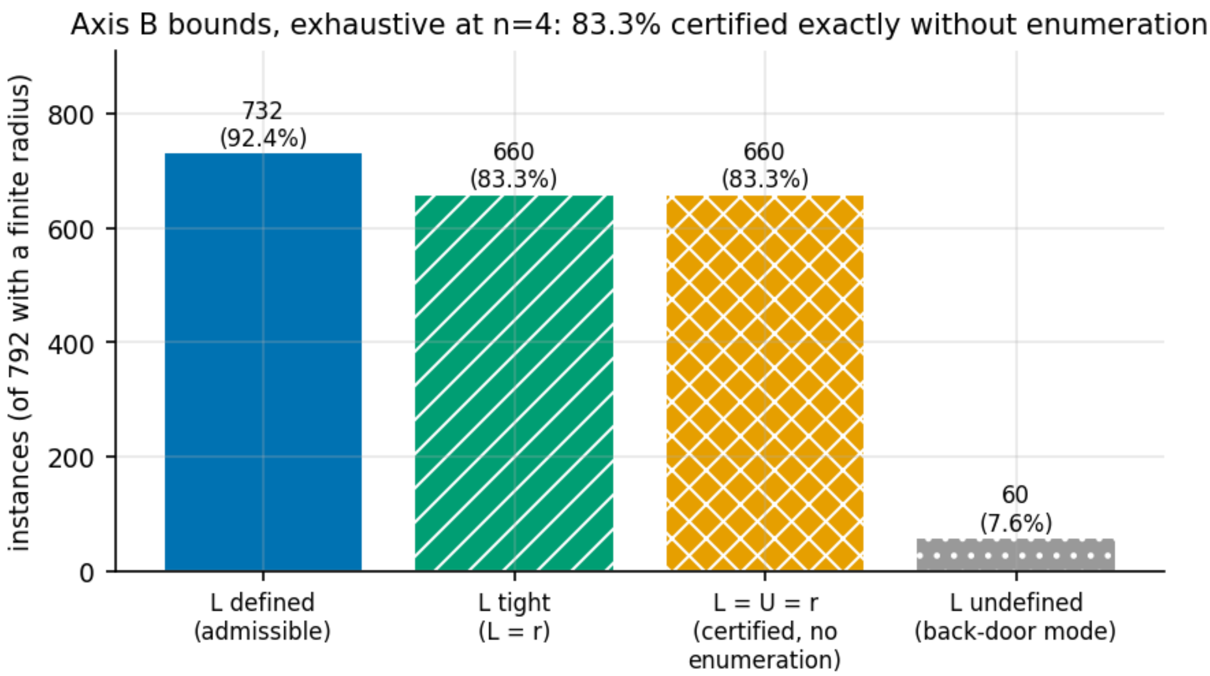


Figure 3. Where the L/U certificate applies. Take from this: five instances in six are certified exactly without enumerating the space — but L is undefined in 7.6%, where the binding failure mode is a back-door path becoming unblocked rather than a descendant appearing.

Limitation, stated rather than buried. L covers only *one* of the two failure modes. In 60 of 792 instances (7.6%) no member of Z can be made a descendant of X, yet the radius is still finite, because the binding mode is a back-door path becoming unblocked — for which no bound is implemented. So L is **not** a complete lower bound on its own, and L = UNREACHED must be read as “this mode yields no bound”, never as infinity. The 83.3% is the rate at which the *pair* certifies exactly, not a claim about L alone.

3.6 Axis B work specified but not done

- **Symmetry reduction** via graph automorphisms and canonical forms (B2.6) — not implemented.
- **Incremental Meek closure** (B2.7) — not implemented; the current closure recomputes from scratch.
- **Declarative SAT/ILP/ASP encoding** (B2.8) — not attempted. This was the item most likely to scale differently, and skipping it is the largest unexplored direction in Axis B.
- **Component decomposition** (B2.2) — not implemented. The space-free search made it less urgent, but it is the natural route to larger graphs.
- These were dropped in favour of the L/U certificate, which looked more valuable per unit of effort. That was a judgement call, recorded in PLAN.md as a revision.

4. Axis A — generalisation to synthetic graph ensembles

4.1 Pre-registration

`results/synth/preregistration.md` was committed **before the first large run** and has not been edited since. It fixes, for each of H1–H6, the predicted direction, the statistic that would test it, and what result would falsify it. It also fixes the analysis rules: radii key off *mean* bias and never the sampled maximum; cost is reported against the knowledge-intersected chordal component rather than n; every result row carries the method that produced it; UNREACHED is never aggregated as a number; and no generator, parameter or epsilon may be changed after seeing results in order to obtain a nicer outcome.

4.2 Generators and knowledge simulation

Component	What was built
Generators	Erdős–Rényi over a random topological order; scale-free (preferential attachment); block/community structure; and a designed decoupled-back-door family with an explicit coupling parameter.
Knowledge simulation	K_true drawn from the true DAG at a chosen knows-fraction, then corrupted by omission, flip, compound (both), or tiered/temporal knowledge, which compels many edges at once and is the realistic case.
Degeneracy gate	Rejects and records a reason: treatment not in or adjacent to an undirected component; empty set trivially valid; no valid adjustment set; no atomic perturbation changes any validity; K_assumed inconsistent; no causal path; Z invalid at G0; G0 not in the space; CPDAG too large for BFS.

4.3 The degeneracy rate is itself a result

The grid was 12,960 points over three generators, n = 6…9, four corruption operators and three corruption rates. **1,064 were accepted (8.2%)**, with zero run errors and zero rows violating the radius convention.

rejection reason	count	% of grid
empty set trivially valid	6,889	53.2%
treatment not in or adjacent to component	3,107	24.0%
no atomic perturbation changes validity	696	5.4%
z not identified	620	4.8%
k assumed inconsistent	341	2.6%
cpdag too large for bfs	160	1.2%
z invalid at g0	83	0.6%
ACCEPTED	1,064	8.2%

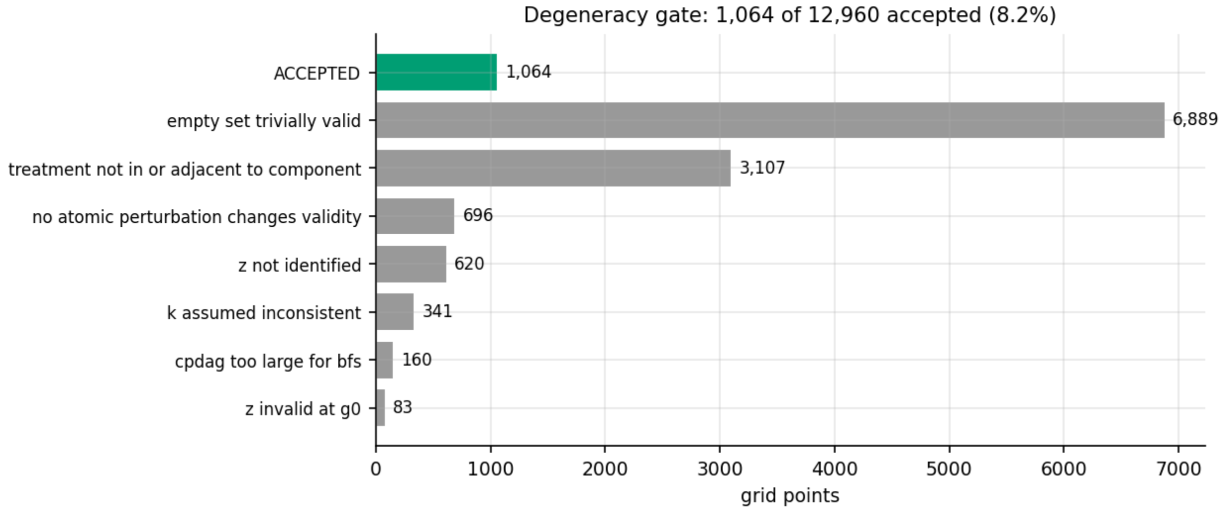


Figure 4. The degeneracy gate. Take from this: roughly nine in ten randomly generated instances are unusable for this study, most often because there is no confounding to speak of. Any ensemble rate quoted without this denominator is misleading — the phenomenon requires fairly specific structure.

4.4 H1 — saturation

The question. Is r_val = 1 the common case? The project's principal risk was that the radius saturates at 1 and therefore carries no information. **Pre-registered prediction:** the mode is 1, with non-trivial mass at 2 and above, and that mass grows as the knowledge-intersected component grows. **Falsified if** more than 90% of non-degenerate instances have r_val = 1.

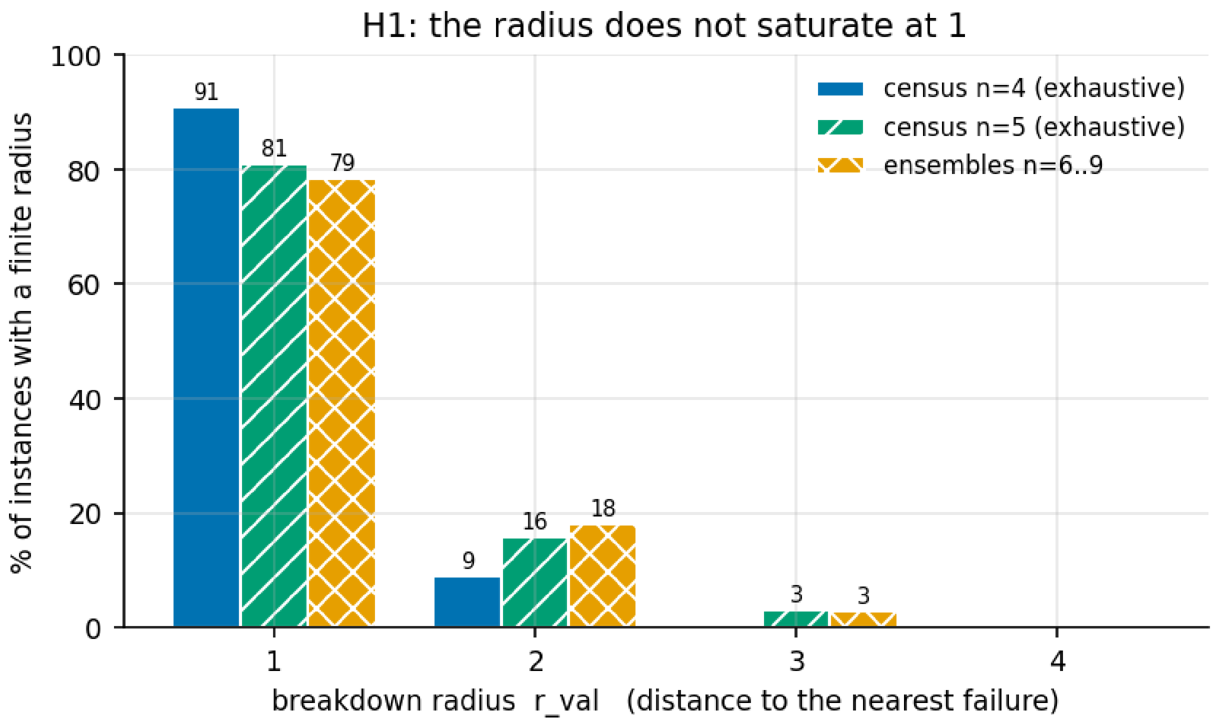


Figure 5. The distribution of the breakdown radius, from two independent lines of evidence: an exhaustive census over every CPDAG on 4 and 5 nodes, and sampled ensembles at n = 6…9. Take from this: r_val = 1 dominates at roughly 80%, but about a fifth of instances have radius 2 or more — well short of the >90% that would have made the radius near-vacuous.

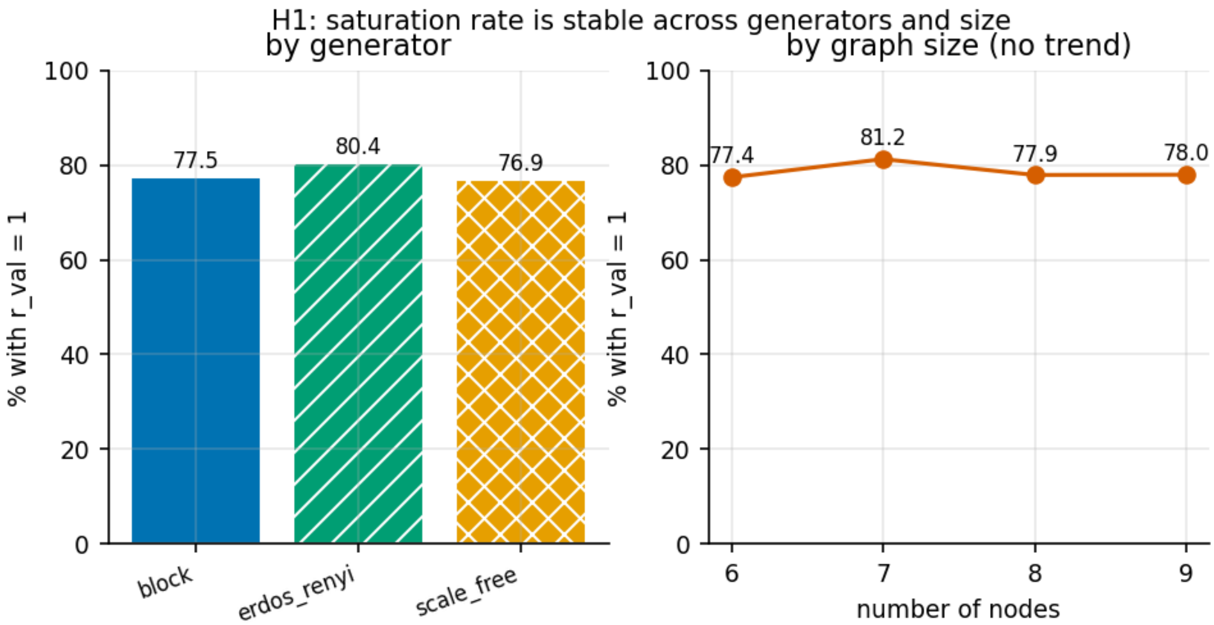


Figure 6. Is that ~80% an artefact of one generator or one size? Take from this: no. It is stable across all three generators (76.9–80.4%) and flat across n = 6…9 — which also means the pre-registered *sub*-prediction, that mass at r ≥ 2 grows with graph size, is NOT supported.

Verdict: H1 is not falsified, but its sub-prediction is. The mode is 1 and roughly 20% of instances have radius ≥ 2, so the radius is not vacuous — this retires the project's principal risk. The claim that mass at $r \geq 2$ grows with the component does not hold: the distribution is essentially flat in n. Reported as a failed prediction rather than omitted.

4.5 H2 and H3 — the frontier. My first answer was wrong.

This is the result I got wrong first, so the correction is given in full rather than quietly replaced.

The wrong version

My initial H2 statistic reported **0 of 134,140** instances with a strictly more robust valid adjustment set — an apparently decisive confirmation that the prior report's flat frontier was a general phenomenon. It was an artefact of my own analysis code, in two compounding ways. First, `h2_frontier` **excluded every comparison in which either radius was UNREACHED**, on the grounds that UNREACHED is not a number. It is not a number — but it is not missing data either. A set that never fails anywhere in the fully enumerated space is *strictly more robust* than one that fails at distance 1, and that is the largest frontier gap there is. Discarding those comparisons discarded precisely the non-flat cases. Second, the census capped candidate adjustment sets at 10 sorted by size, which can hide a more robust larger set.

The corrected version

Exhaustive at n=4 and n=5, no cap, and with UNREACHED ordered *above* every finite radius:

	n = 4	n = 5	total
instances (≥2 valid sets, O* defined)	2,652	247,080	249,732
frontier NON-FLAT	816 (30.8%)	53,880 (21.8%)	54,696 (21.9%)
O* STRICTLY BEATEN	0	0	0 (0.00%)
O* never fails anywhere	1,584	147,740	149,324

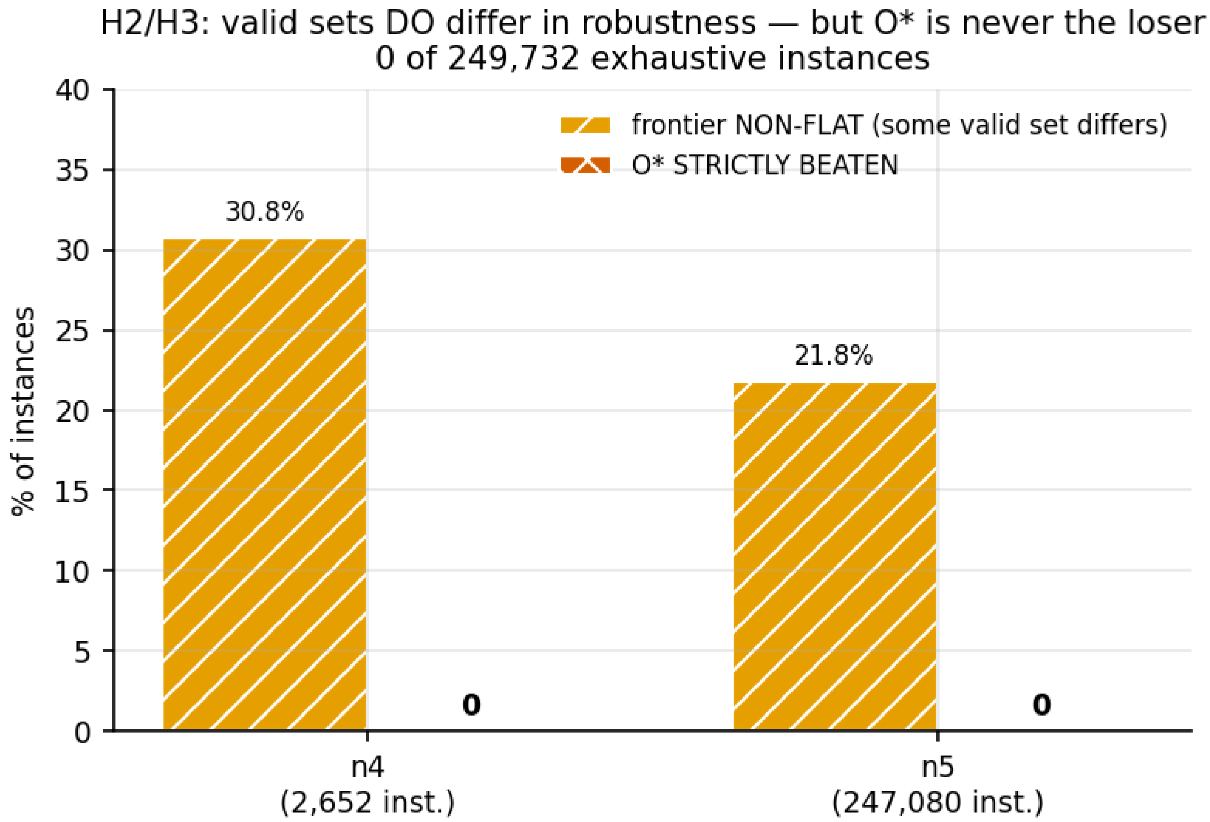


Figure 7. The corrected frontier statistic. Take from this: valid adjustment sets genuinely DO differ in robustness — so the prior report's flat frontier is not general — yet across a quarter of a million exhaustive instances the optimal set is never the less robust choice.

The conclusion changes, and improves. The frontier is non-flat in 21.9% of instances, so valid adjustment sets really do differ in robustness. But the optimal set is never strictly beaten. Therefore: **there is no efficiency–robustness tradeoff — the most efficient adjustment set is also among the most robust**. That is a stronger and more actionable statement than either “the frontier is flat” or “a tradeoff exists”: an analyst choosing O* gives up nothing in robustness.

Excluding the empty adjustment set — trivially robust, since an empty set cannot acquire a descendant of the treatment, and gated out of the ensembles anyway — the n=4 non-flat rate is 10.96% of 2,628 instances, and O* is still never beaten. Both figures are reported because the choice is material.

H3 — the designed families, and a structural obstruction

The pre-registered plan was to construct graph families with two back-door routes whose blocking sets are disjoint and lie in different chordal components, so that a perturbation confined to one component cannot invalidate the sets blocking via the other. This turned out to be **impossible for the construction shape attempted**, and the reason is worth recording. Identification requires X → Y to be compelled, which requires a v-structure at Y; adding the node that supplies it also freezes the confounder→X and confounder→Y edges, which makes O* structurally invariant across the whole space — so no atomic perturbation changes any validity status and every instance is gated out. The family went from “zero valid adjustment sets” to “100% gate-rejected” for the opposite reason.

There is a genuine tension between **identification** (which needs compelled edges around the target) and **perturbability** (which needs undirected ones). I did not find a construction satisfying both and I am not claiming none exists. This did not block H3, because the corrected H2 analysis shows designed adversarial families are unnecessary: roughly a fifth of *ordinary* small instances already exhibit a non-flat frontier. The designed experiment was superseded by an exhaustive one, which is better evidence anyway.

4.6 H4 — the naive baseline. NOT RUN.

The naive K-count radius was **not computed for the ensembles**. The runner's schema carries no naive_radius column, and I chose to spend the remaining effort on correcting the H2 statistic instead. The prior report's single-example finding — that the naive count understates the model radius, off by exactly one in all three scenarios, with 29% of elements disagreeing by two hops or more — is therefore **neither confirmed nor refuted** by this session. It is the cheapest remaining item: the machinery already exists in `bkrobust.demo.baseline` and the runner needs one extra column.

4.7 H5 — the middle regime

The question. How often does r_val come out low while r_eps is high — fragile identification with a stable estimate? This determines whether r_eps deserves to carry part of the project's contribution. **Pre-registered prediction:** 20–50% of non-degenerate instances.

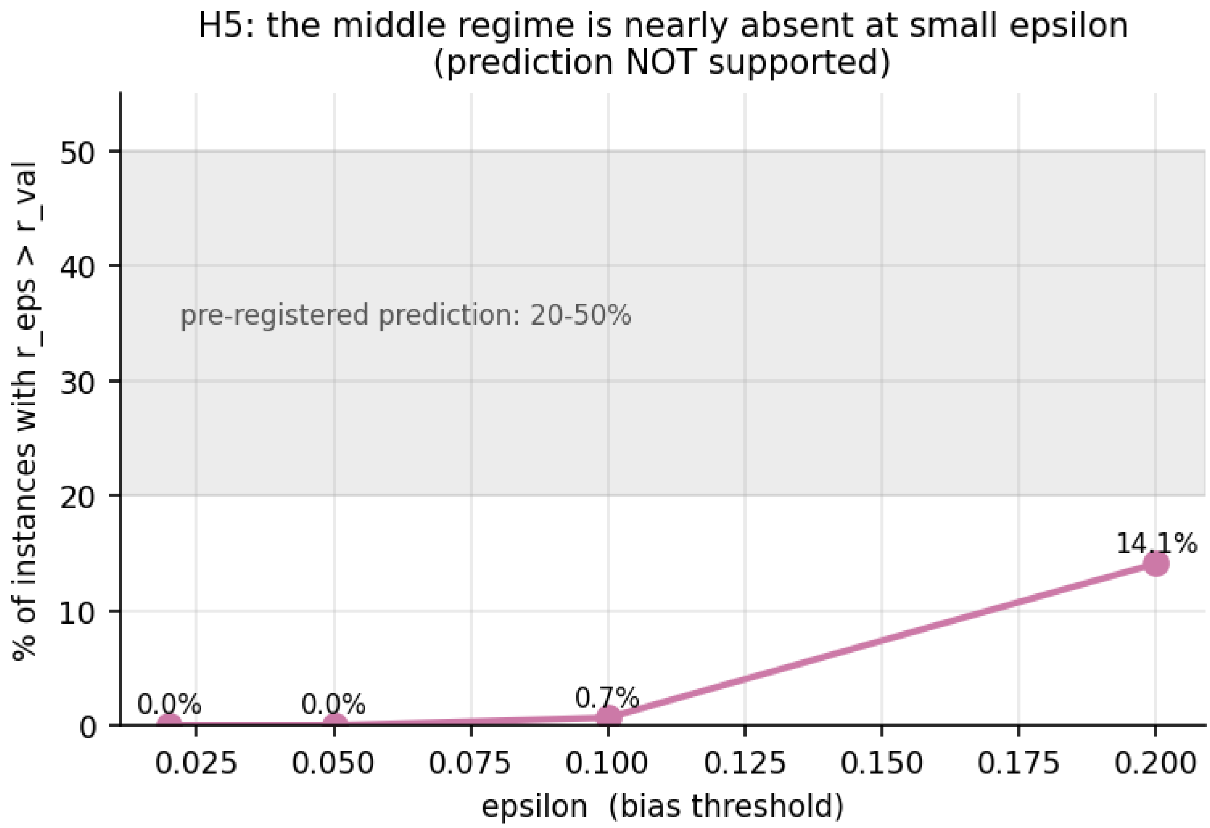


Figure 8. The middle regime against the bias threshold, over 1,050 ensemble instances; the shaded band is the pre-registered prediction. Take from this: at small epsilon the middle regime essentially does not occur, because bias sits at machine zero while Z stays valid and jumps as soon as it does not. Only at epsilon = 0.2 — a fifth of the effect size — does it reach 14.1%.

Verdict: the prediction is not supported. The implication for the project is concrete: r_eps carries much less independent information than the single worked example suggested. It is nearly redundant with r_val unless epsilon is set at a substantial fraction of the effect size.

4.8 H6 — calibration

Coverage below 1.00 would be a **bug**, not a finding — the pre-registration says so and the run halts on it. Conservativeness measures how pessimistic the certificate is: among graphs lying outside the certified radius, how many nevertheless admit Z as valid.

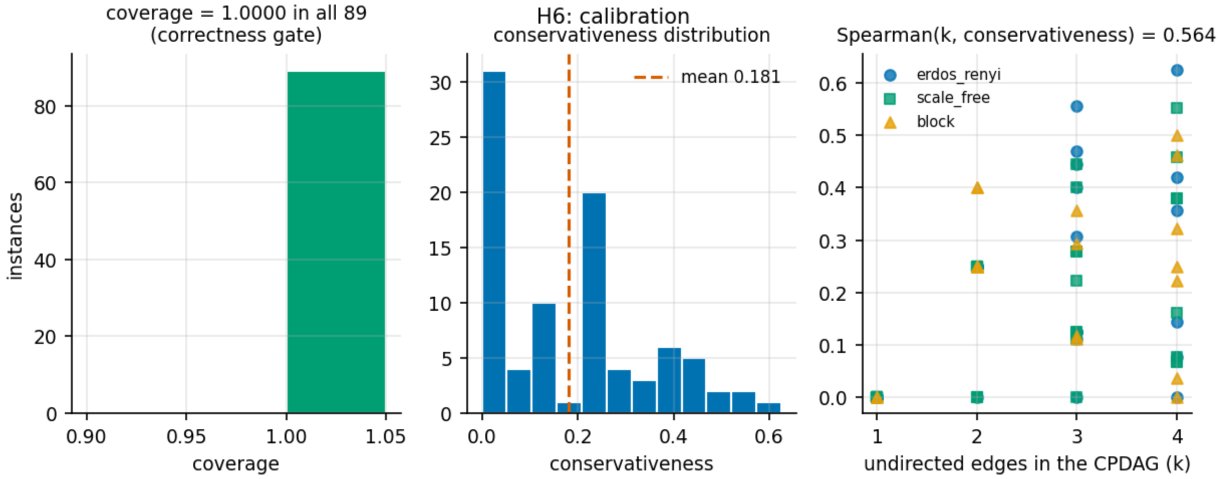


Figure 9. Calibration over 89 instances. Take from this: coverage is exactly 1.0000 everywhere, as it must be — the correctness gate passes. Conservativeness averages 0.181, well below the prior report's single-example 0.43–0.47, and it is predicted by the size of the knowledge-intersected component (Spearman 0.564), which is what H6 anticipated.

Statistic	Value
instances	89
coverage (minimum across all instances)	1.0000
coverage = 1.0000 in every instance	YES
conservativeness mean	0.181
conservativeness range	0.000 – 0.625
Spearman(component size, conservativeness)	0.564

5. Bugs found, and how each was caught

House style inherited from the prior report: record every defect, including my own, with the mechanism that surfaced it. Five of the nine below are mine. Bug 2 is the most consequential — it would have put a confident false claim in the report — and it was caught only by re-deriving a result I had already written down.

#	Origin	Bug	What went wrong	How it was caught / resolved
1	MINE	.gitignore silently excluded results/synth/ and results/search/	The /results/* rule from the earlier session exempted only the demo directory, so a commit whose message claimed to add the Axis B differential rows added nothing at all, and the pre-registration would have gone the same way.	git commit reported “nothing to commit” for a brand-new file. Corrected in LOG.md; the numbers were unaffected, only the claim about where the file landed.
2	MINE	The H2 statistic discarded UNREACHED comparisons — the most consequential bug	h2_frontier excluded every comparison in which either radius was UNREACHED, and the census capped candidate sets at 10. Together these produced a confident and wrong “0 of 134,140 instances have a more robust valid set”, which would have put a false claim in the report.	Re-deriving the frontier from scratch without the cap and getting 23% non-flat. The corrected statistic is in section 4.5.
3	MINE	An O(|space|²) check run once per combination	check_conjecture1 is quadratic in the space size and was being called for every (G0, X, Y, Z) combination. On a 120-element space with ~19k combinations that is ~276M pair comparisons for a single CPDAG.	The n=5 sweep stalled at the same CPDAG index on two independent runs. Conjecture 1 is a theorem, so the check guards the implementation and a sample suffices; it is now sampled at a recorded rate.
4	MINE	The space-size guard ran after the cubic covering-relation build	build_space computes the covering relation, which is cubic in the element count, before the study could reject an over-large space — so the cost was paid before the rejection.	Same investigation as bug 3. Elements are now enumerated first (cheap) and covers built only if the size passes.
5	MINE	I pasted a garbled graph into a subagent brief	The edge string I sent contained both V1→V6 and V6→V1, which the MPDAG constructor provably forbids. No such graph can exist; the paste was mine, not the code's.	The subagent spotted the contradiction, declined to trust it, and independently reconstructed the mechanism as a minimal 4-node example — the better artefact. The underlying phenomenon was real and had been independently measured beforehand.
6	INHERITED	Meek(Ĉ, K) can fall outside enumerate_space (0.09% of closures)	A graph that is Meek-closed, keeps every compelled edge, satisfies [G] ⊆ [Ĉ] and represents 5 real DAGs is nonetheless excluded from the space. The cause is that chordality is checked on the undirected subgraph alone, so a component whose only chord is a DIRECTED edge reads as chordless.	A 12,960-point run crashed; incremental writing preserved 377 rows. Measured at 2/2235 closures. Deliberately gated rather than fixed: relaxing the validity definition would invalidate every result resting on the current enumeration, including the ~2M-comparison conjecture sweep. Carried to NEXT.md.
7	DELEGATED	9% of accepted instances had r_val = 0, which the convention forbids	All had an empty Z. The runner proposed target pairs with no causal path from X to Y, so the causal-node set is empty, O* is empty, and the empty set is not a valid adjustment set because a back-door path is open.	Inspecting the output distribution rather than trusting it. conventions.py had already anticipated this — r = 0 is DEGENERATE and must be gated at generation — so the convention was right and the gate was incomplete. Two new gate reasons added.
8	DELEGATED	The H3 family had ZERO valid adjustment sets for its designed target pair	The session's most important experiment was silently untestable: X−Y came back undirected in the CPDAG, so in some extensions the arrow runs backwards and the effect is not identified at all.	Reviewing the generator against its stated purpose before running it, rather than running it and reading the output. Sent back with a diagnosis and a fix.
9	DELEGATED	The H3 fix then froze O*, making the family 100% gate-rejected the other way	Forcing X → Y to be compelled requires a v-structure at Y, and the node supplying it also freezes the confounder edges, making O* structurally invariant so that no atomic perturbation changes any validity status.	Found and honestly reported by the subagent itself, which flagged it as a genuine mathematical tension rather than a residual bug. See section 4.5.

One issue deliberately left open. Bug 6 points at a real tension in the inherited MPDAG validity definition. Resolving it means deciding whether the validity definition or the chordality test is wrong, and then re-running everything that depends on the space enumeration. That should happen before scaling up, not after — it is item 3 in NEXT.md.

6. Orchestration: what was delegated and what came back

One subagent was used, for the Axis A generators, knowledge simulation and instance runner — a well-specified, independently verifiable unit with disjoint file ownership. Everything else (the convention, the frozen core, the conjecture machinery, the accelerated search, the bounds, the census, the frontier analysis, the hypothesis analysis, the figures and this report) was done in-house, because the brief reserved design, prioritisation, integration and scientific judgement for the orchestrator.

The deliverable was **reviewed, not trusted**, and sent back three times:

- **Round 1 — the H3 family was unusable.** I checked the generator against its purpose and found zero valid adjustment sets for the designed (X, Y) pair, with the diagnosis that X−Y was left undirected so the effect was not identified. Sent back with the fix (add an outcome-only parent to compel X → Y) and sharp acceptance criteria.
- **Round 2 — run_grid crashed instead of gating.** A rare case (0.09%) where a legitimate Meek closure falls outside the enumerated space killed a 12,960-point run. Sent back requiring a g0_not_in_space gate reason plus general per-instance error resilience, and explicitly forbidding any change to the frozen core.
- **Round 3 — the gate admitted r_val = 0.** Found by inspecting the output distribution. Sent back requiring two new gate reasons and a test asserting the convention over a sample rather than one case.
- **Independently verified after acceptance:** the other three generators were checked myself for the same degeneracy (they were healthy — 40/40 with ≥2 valid adjustment sets), and the subagent's hand-rolled Spearman implementation from the prior session was cross-checked against scipy to <1e-9 once scipy was installed.

The subagent also returned one finding I had not asked for and could not have obtained by review: that its own H3 fix created the identification-vs-perturbability tension described in section 4.5. It reported this as a limitation of its own work rather than presenting the fix as a success, which is the behaviour that made the delegation worth it.

7. What this does not show

- **Standing assumptions.** Fixed skeleton, causal sufficiency, faithfulness and oracle conditional-independence testing are assumed throughout. Finite-sample skeleton error is entirely outside this analysis and may well dominate everything measured here in practice.
- **Small graphs.** Exhaustive results cover n ≤ 5; the ensembles reach n = 9. Nothing here establishes behaviour at realistic sizes.
- **Conjecture 2 is not proved.** ~2M comparisons without a counterexample is strong evidence, not a proof, and the densest 2.2% of n=5 CPDAGs were not examined.
- **L is not a complete lower bound.** It covers one failure mode; 7.6% of instances fail by the other, for which no bound is implemented.
- **The 83.3% certificate rate is an n=4 number** and is not established at larger sizes.
- **Bias is a sampled proxy.** Radii key off mean bias; the maximum is a sampled statistic, measurably unstable, and is never quoted as a bound.
- **H4 was not run at all.**
- **Ensemble-specific vs general.** H1's ~80% and H2's “O* is never beaten” hold across all three generators AND the exhaustive census, so they are not artefacts of one family. H5's epsilon-dependence and H6's conservativeness levels are ensemble-specific and should not be quoted as general.
- **“O* is never beaten” is an empirical regularity, not a theorem.** It rests on 249,732 exhaustive instances at n = 4 and 5. A single counterexample would matter, and it has not been tested above n = 5 or on dense CPDAGs.

8. Reproduction

Test suite (277 passing):

```
PYTHONPATH=src python3 -m pytest tests/core tests/search tests/synth tests/demo -q
```

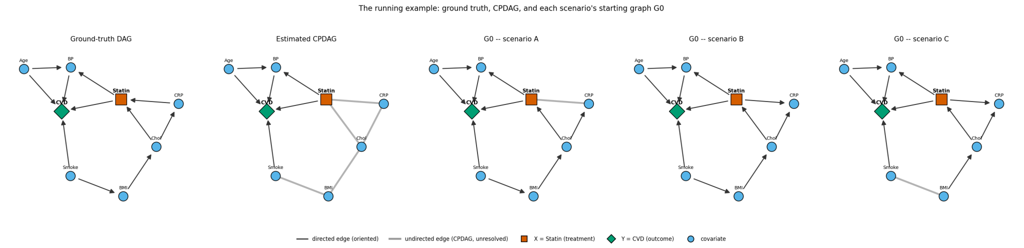
Key runs, all seeded from root seed 20260919:

```
PYTHONPATH=src python3 -c "from bkrobust.search.conjecture_study import run_study;
run_study(5,'results/search/conjectures')"
PYTHONPATH=src python3 -c "from bkrobust.search.differential import differential_sweep;
differential_sweep(4)"
PYTHONPATH=src python3 -c "from bkrobust.search.radius_census import census; census(5)"
PYTHONPATH=src python3 -c "from bkrobust.search.frontier import frontier_sweep; frontier_sweep(4)"
PYTHONPATH=src python3 -c "from bkrobust.analysis.session_figures import build_all; build_all()"
PYTHONPATH=src python3 -c "from bkrobust.analysis.session_pdf import build; build()"
```

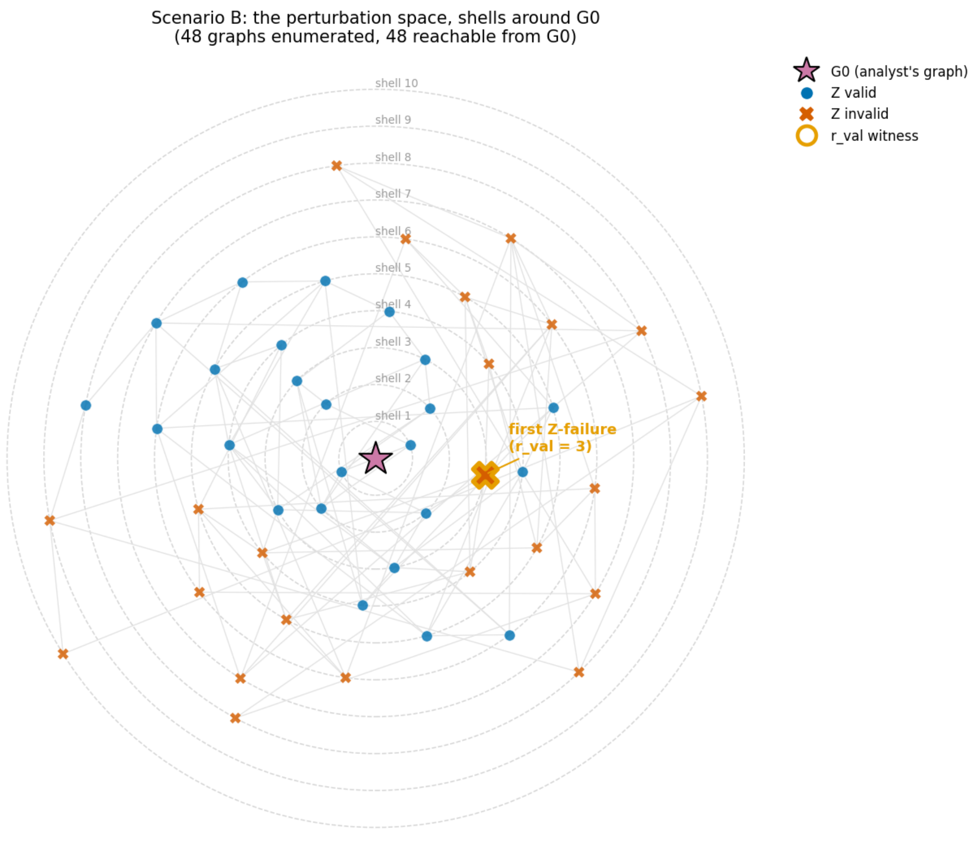
Runtimes. n=5 conjecture sweep 926 s; n=5 census 107 s; Axis A grid 132 s; differential sweep 2.5 s; frontier n=4+n=5 ~20 min; figures ~5 s; this PDF ~2 s.

Appendix A — figures from the prior session (worked example)

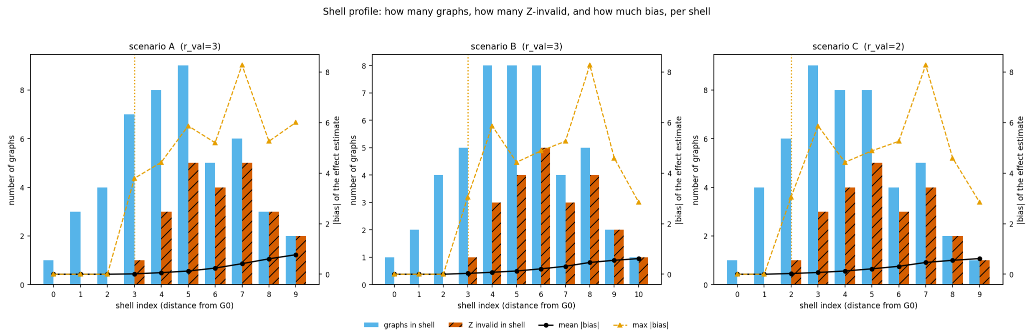
These are **not** from this session. They come from the earlier breakdown-radius demonstration on a single 8-variable clinical example (statin therapy → cardiovascular events) and are reproduced here so this document is the single centralised record. That example is what this session's ensembles generalise, and its flat frontier is what section 4.5 corrects.



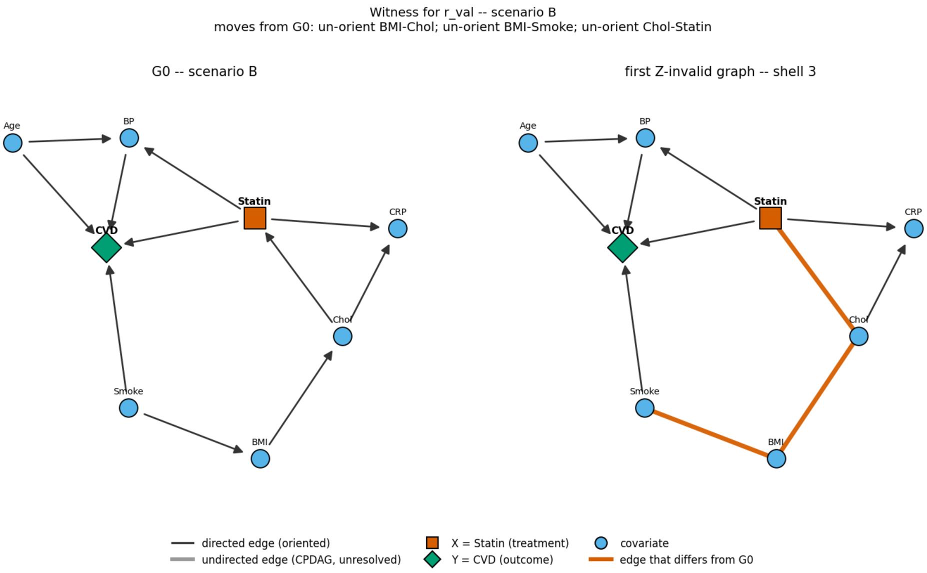
A1. The worked example: ground-truth DAG, the estimated CPDAG, and G0 for the three analyst knowledge states. The treatment sits inside the undirected component, so the data alone cannot say whether CRP precedes or follows treatment.



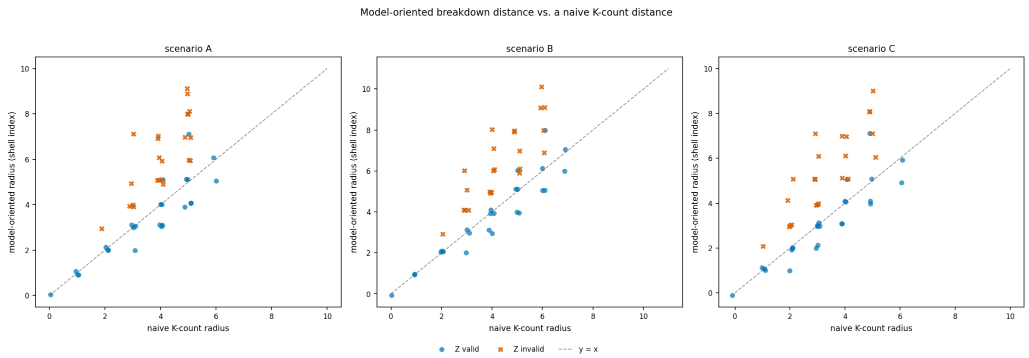
A2. The perturbation space laid out by shell index around G0. Failures begin at a definite radius and grow outward; the inner two shells are clean.



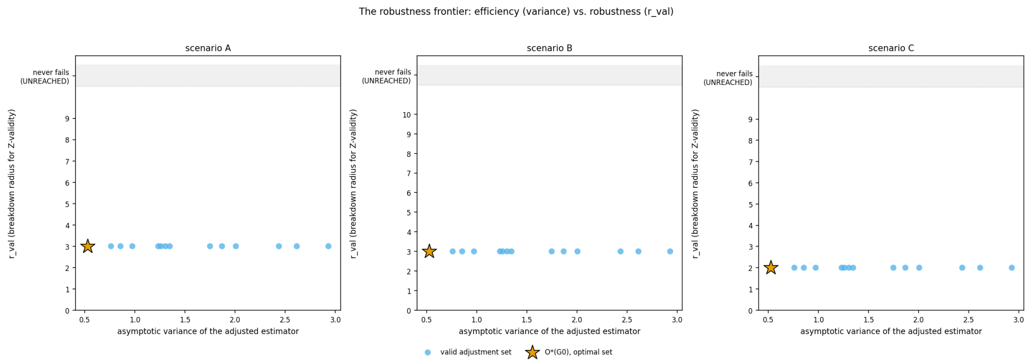
A3. Shell index against graph counts and bias. Bias is machine-zero while Z stays valid and rises only once validity is lost.



A4. G0 beside the first failing graph. The failure is reached by retracting three claims, not by asserting anything false.



A5. Model-oriented radius against the naive K-count radius. Monotone but loose; the naive count reads as “closer” than the graph really is. This is the comparison H4 would have generalised — and H4 was not run this session.



A6. The prior session's robustness frontier, which looked flat on this one example. Section 4.5 shows that flatness is NOT general: across 249,732 exhaustive instances the frontier is non-flat in 21.9% — though O* is still never the loser.

Appendix B — file inventory

Path	Purpose
PLAN.md	Living plan with six recorded revisions and their reasons.
LOG.md	Append-only audit trail: what was launched, returned, integrated, discarded, and every bug.
report_synth_and_search.md	The markdown session report.
NEXT.md	What is now safe to claim, what is not, and the next experiments.
SESSION_REPORT.pdf	This document.
results/synth/preregistration.md	H1–H6 fixed before the first large run; unedited since.
src/bkrobust/core/	The frozen shared core (5 modules).
src/bkrobust/search/	Conjectures, exhaustive study, exact search, differential testing, census, frontier.
src/bkrobust/synth/	Generators, knowledge simulation, instance runner (delegated, reviewed).
src/bkrobust/analysis/	Hypothesis statistics, session figures, this PDF builder.
results/search/	Conjecture sweeps, differential rows, bounds, speedup.
results/synth/	Census, main grid, frontier, calibration, pilot.
figures/sess_f*.{pdf,png}	This session's nine figures.
tests/core/, tests/search/, tests/synth/	Test suites for the new work.